

Lista de Exercícios Evolutivos – POO - 01

Exercício 1 – Classe Pessoa

Crie uma classe chamada Pessoa com os seguintes atributos:

- nome
- idade

Tarefas:

- Criar os atributos privados.
- Criar os métodos get e set.
- Criar um método `exibirDados()` que mostre o nome e a idade da pessoa.

No programa principal:

- Crie um objeto.
 - Defina valores usando set.
 - Exiba os dados usando o método criado.
-

Exercício 2 – Classe Produto

Crie uma classe chamada Produto com os atributos:

- nome
- preco

Tarefas:

- Encapsular os atributos.
- Criar getters e setters.
- Criar um método `mostrarProduto()` que exiba nome e preço.

No programa principal:

- Cadastre dois produtos.
 - Mostre os dados de ambos.
-

Exercício 3 – Classe Aluno

Crie uma classe chamada Aluno com os atributos:

- nome
- matricula
- nota1
- nota2

Tarefas:

- Criar getters e setters.
- Criar um método calcularMedia() que retorne a média das duas notas.
- Criar um método verificarAprovacao() que informe se o aluno foi aprovado considerando média mínima 7,0.

No programa principal:

- Cadastre um aluno.
 - Defina os valores.
 - Mostre média e situação.
-

Exercício 4 – Classe Retângulo

Crie uma classe chamada Retangulo com os atributos:

- largura
- altura

Tarefas:

- Encapsular os atributos.
- Criar getters e setters.
- Criar um método calcularArea().
- Criar um método calcularPerimetro().

No programa principal:

- Criar um retângulo.
- Informar largura e altura.
- Exibir área e perímetro.

Exercício 5 – Classe ContaBancaria

Crie uma classe chamada ContaBancaria com os atributos:

- numeroConta
- titular
- saldo

Tarefas:

- Criar getters e setters.
- Criar um método depositar(valor).
- Criar um método sacar(valor).
- Criar um método consultarSaldo().

Regra:

- O saque só pode ocorrer se houver saldo suficiente.

No programa principal:

- Criar uma conta.
 - Fazer depósito.
 - Fazer saque.
 - Mostrar saldo final.
-

Exercício 6 – Classe Carro

Crie uma classe chamada Carro com os atributos:

- marca
- modelo
- velocidade

Tarefas:

- Criar getters e setters.
- Criar um método acelerar() que aumente a velocidade.
- Criar um método frear() que reduza a velocidade.
- Criar um método exibirVelocidade().

Regra:

- A velocidade não pode ficar negativa.

No programa principal:

- Criar um carro.
- Simular aceleração e frenagem.
- Mostrar o valor final da velocidade.

Exercício 7 – Classe Funcionário

Crie uma classe chamada Funcionario com os atributos:

- nome
- cargo
- salario

Tarefas:

- Encapsular os atributos.

- Criar getters e setters.
- Criar um método `aumentarSalario(percentual)`.
- Criar um método `exibirFuncionario()`.

No programa principal:

- Criar um funcionário.
- Mostrar os dados antes e depois do aumento.

Exercício 8 – Classe Livro

Crie uma classe chamada Livro com os atributos:

- `titulo`
- `autor`
- `quantidadePaginas`
- `emprestado` (verdadeiro ou falso)

Tarefas:

- Criar getters e setters.
- Criar um método `emprestarLivro()`.
- Criar um método `devolverLivro()`.
- Criar um método `verificarDisponibilidade()`.

Regra:

- Um livro já emprestado não pode ser emprestado novamente até ser devolvido.

No programa principal:

- Criar um livro.
 - Realizar empréstimo.
 - Tentar emprestar novamente.
 - Devolver e verificar disponibilidade.
-

Exercício 9 – Classe Celular

Crie uma classe chamada Celular com os atributos:

- marca
- modelo
- nivelBateria

Tarefas:

- Encapsular os atributos.
- Criar getters e setters.
- Criar um método carregarBateria(valor).
- Criar um método usarBateria(valor).

Regras:

- A bateria não pode passar de 100.
- A bateria não pode ficar abaixo de 0.

No programa principal:

- Criar um celular.
- Simular uso e recarga.
- Mostrar o nível final de bateria.

Exercício 10 – Classe Filme

Crie uma classe chamada Filme com os atributos:

- titulo
- genero
- duracao
- avaliacao

Tarefas:

- Criar getters e setters.
- Criar um método `exibirFichaTecnica()`.
- Criar um método `alterarAvaliacao(novaAvaliacao)`.

Regra:

- A avaliação deve estar entre 0 e 10.

No programa principal:

- Criar um filme.
- Mostrar os dados.
- Alterar a avaliação.
- Mostrar novamente.

Exercício 11 – Classe Paciente

Crie uma classe chamada Paciente com os atributos:

- nome
- idade
- peso
- altura

Tarefas:

- Criar getters e setters.
- Criar um método `calcularIMC()`.
- Criar um método `classificarIMC()`.

No programa principal:

- Criar um paciente.
 - Exibir o IMC calculado.
 - Exibir a classificação.
-

Exercício 12 – Classe ReservaHotel

Crie uma classe chamada ReservaHotel com os atributos:

- nomeHospede
- quantidadeDias
- valorDiaria

Tarefas:

- Encapsular os atributos.
- Criar getters e setters.
- Criar um método calcularTotalHospedagem().
- Criar um método exibirReserva().

No programa principal:

- Criar uma reserva.
 - Mostrar os dados e o valor total.
-

Exercício 13 – Classe Jogador

Crie uma classe chamada Jogador com os atributos:

- nome
- pontuacao
- nivel

Tarefas:

- Criar getters e setters.
- Criar um método adicionarPontos(valor).
- Criar um método subirNivel().

Regra sugerida:

- A cada 100 pontos, o jogador sobe 1 nível.

No programa principal:

- Criar um jogador.
 - Simular ganho de pontos.
 - Exibir pontuação e nível.
-

Exercício 14 – Classe AgendaContato

Crie uma classe chamada AgendaContato com os atributos:

- nome
- telefone
- email

Tarefas:

- Criar getters e setters.
- Criar um método `exibirContato()`.
- Criar um método `atualizarTelefone(novoTelefone)`.

No programa principal:

- Criar um contato.
 - Exibir os dados.
 - Atualizar o telefone.
 - Exibir novamente.
-

Exercício 15 – Sistema de Biblioteca Simplificado

Crie uma classe chamada LivroBiblioteca com os atributos:

- titulo
- autor
- codigo

- disponível

Tarefas:

- Criar getters e setters.
- Criar um método emprestar().
- Criar um método devolver().
- Criar um método exibirInformacoes().

Regras:

- O livro só pode ser emprestado se estiver disponível.
- Ao devolver, ele volta a ficar disponível.

No programa principal:

- Cadastrar pelo menos 3 livros.
- Simular empréstimos e devoluções.
- Exibir o estado final de cada livro.

Resultados devem ser registrados no Github de cada aluno